

Design Document

Primap (Web Application)

Document Control

Document Information	
Title	Design Document for Primap (Web Application)
Date	February 20th, 2026
Version	1.0
Prepared for	CS3213 (Foundations of Software Engineering), National University of Singapore
Authors	Group 4 • Nicholas Tang Boon Keat • Lim Kai Xiang Sean • Kim Jae Hyeok • Ha Jiwoon

Version History

Version	Date	Description of Changes
1.0	Feb 20th, 2026	Initial submission

1. Project Overview and Goals

1.1 Product Perspective

Primap is a standalone web application designed to replace the current spreadsheet-based workflow used by the Raffles' Banded Langur Working Group (RBLWG). It integrates volunteer onboarding (self-registration with admin approval), survey round and walk slot management, and observation reporting into a single system. The system supports real-world field conditions by enabling offline draft creation and synchronisation when connectivity is restored.

Primap relies on external services for:

- Authentication and session management
- Persistent relational storage for data
- Media storage for photos and videos attached to reports
- Email notifications for reminders and cancellation alerts
- Map visualisation of observation locations

1.2 Design Goals and Objectives

The design aims to achieve the following goals:

1. **Reliable offline reporting**

Volunteers can create and save drafts offline and synchronise later without data loss.

2. **Data privacy and ethical protection of wildlife**

Observation locations and outcomes are not publicly accessible. Access to reports is granted based on users' roles and whether they created the report.

3. **High usability across diverse users**

The system supports fast and easy sign-up and reporting as well as clear, simple, forms.

4. **Admin features and data export**

For admin users, the system supports scheduled bulk slot creation, volunteer analytics, and exporting of data.

2. Quality Attributes for the System

2.1. Reliability (Offline-first Drafting & Synchronisation)

Primap must support offline creation of draft observations and support data synchronisation once network connectivity is regained. Offline capability is essential because survey routes may have poor cellular coverage and volunteers often need to record information whilst on walks. Volunteers also often only submit reports after walks.

Rationale

Reliability is critical because “Not Sighted” and “Sighted” observations are both scientifically valuable, and loss of drafts due to connectivity issues would reduce data completeness and reliability of the system.

Quality Attribute Scenario Components

- **Stimulus Source:** Volunteer using a mobile device during a survey walk.
- **Stimulus:** Creates/edits a draft observation while offline, then later reconnects and triggers sync.
- **Artifact:** PWA Offline Store + Sync Engine + Supabase (DB/Storage) + Edge Functions/DB Functions.
- **Environment:** No network during the walk with network connectivity after
- **Response:** System persists drafts locally and queues user operations. Once network connectivity is restored, the system syncs the data with the database while preventing duplicate submissions using idempotency keys
- **Response Measure:** In $\geq 99\%$ of tested offline/online transition scenarios, drafts and submissions synchronise with no data loss and no duplicate final submissions.

2.2. Privacy & Ethics (Role/Slot-Based Access Control)

Primap must prevent real-time or near real-time disclosure of primate locations by ensuring observation data is only accessible to authorised users, especially during active rounds.

Rationale

A key requirement is the restriction of observation details during active rounds to protect wildlife. Privacy is also required due to PDPA constraints on volunteer personal information.

Quality Attribute Scenario Components

- **Stimulus Source:** Authenticated but volunteer did not take part in the slot, or unauthenticated user
- **Stimulus:** User attempts to access observation details (including GPS coordinates and outcomes) for a slot they are not assigned to
- **Artifact:** Authorisation enforcement layer (Supabase Postgres Row-level security (RLS) + Edge Function checks)
- **Environment:** Active survey round (Open)

- **Response:** System rejects the request and does not disclose observation details.
- **Response Measure:** 100% of unauthorised attempts are denied.

2.3. Usability (Fast & Simple Volunteer Workflows)

Primap must be easy for new or less tech-savvy volunteers to learn and use. Basic tasks, including signing up for slots and submitting observations, should be intuitive to understand and easy to navigate, with clear error feedback.

Rationale

Volunteers vary widely in technical proficiency and the usability of the system directly impacts participation rates and the quality of reports.

Quality Attribute Scenario Components

- **Stimulus Source:** New volunteer.
- **Stimulus:** Volunteer attempts to sign up for a walk slot and submit an observation.
- **Artifact:** Volunteer UI flows (slot listing, reporting forms, guidance).
- **Environment:** Typical mobile device and network; first-time usage.
- **Response:** System provides clear step-by-step forms, with confirmation messages and clear error dialogs if needed.
- **Response Measure:** $\geq 80\%$ of new volunteers complete sign-up + submit within 10 minutes without external help.

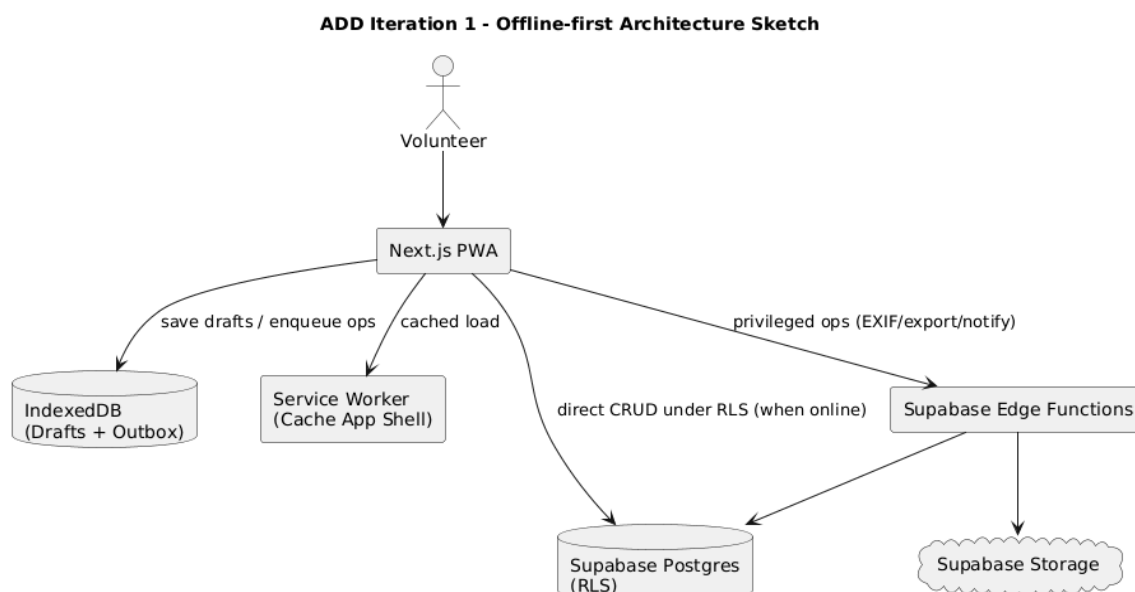
3. Architecture and High-Level Design

3.1. Iterative Architectural Design

1: Reliability (Offline-first Drafting & Synchronisation)

- **Architectural Driver:** The system shall allow offline draft creation and synchronise without data loss in $\geq 99\%$ tested sync scenarios.
- **Approaches:**
 - **Option A (Local Storage only):** Store drafts in local storage which can then be submitted when the user is online.
 - **Option B (Offline-first PWA):** Cache PWA and store drafts in IndexedDB. Queue operations to be synced when network connectivity is regained.
- **Tradeoffs**
 - **Option A**
 - **Pros:** Less client complexity
 - **Cons:** Even if the draft is saved, closing the browser or tab will require network access to load the webpage again to edit the draft, resulting in a more finicky and unreliable user experience.
 - **Option B**
 - **Pros:** Better UX under poor network conditions
 - **Cons:** Requires sync queue and a more complicated design to handle syncing of data and prevent duplicate submissions. Higher implementation complexity to configure service workers and caching
- **Resulting Decisions**

We will adopt **Option B (Offline-first PWA)** as the offline drafting is an important requirement and ensuring a better UX and reliable syncing of data is worth the added complexity.

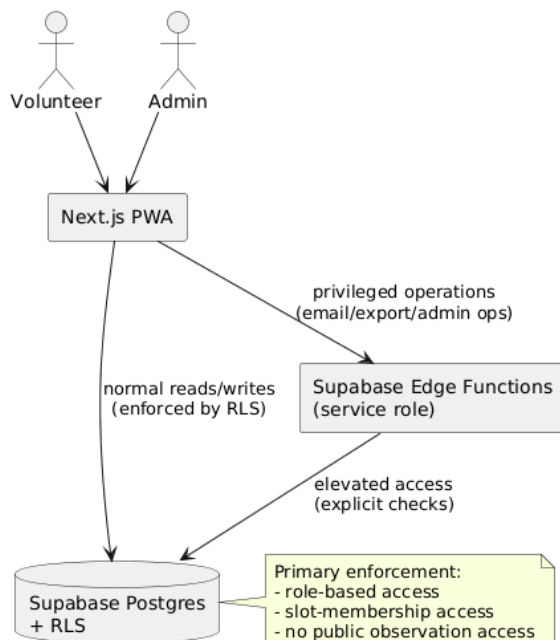


2: Privacy & Ethics (Access Control)

- **Architectural Driver:** During an active round, observation details shall be accessible only to volunteers assigned to the relevant slot and admins.
- **Approaches:**
 - **Option A (Application-only Authorisation):** Enforce access rules and checks only in API routes.
 - **Option B (Database-enforced Authorisation using RLS):** Enforce access constraints in the DB via RLS and use Edge Functions for privileged operations and defense-in-depth checks.
- **Tradeoffs**
 - **Option A**
 - **Pros:** Faster initial implementation
 - **Cons:** Higher security risk if API bug exists and weaker defense-in-depth
 - **Option B**
 - **Pros:** Stronger security boundary and reduces impact of API mistakes
 - **Cons:** Requires careful schema/RLS design and testing
- **Resulting Decisions**

We will adopt **Option B (Database-enforced Authorisation using RLS)** for a more robust privacy enforcement measure as we protect the data in multiple layers, including the lowest database layer, ensuring data is only accessible to the relevant parties.

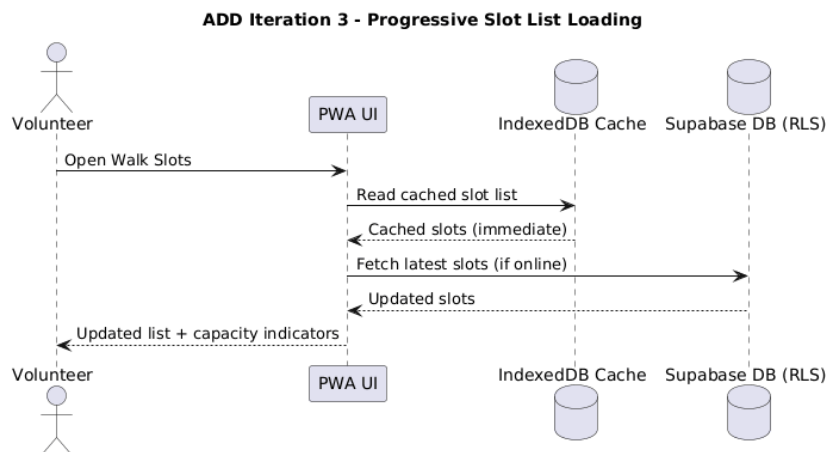
ADD-2 Security Boundary Sketch (RLS + Edge Functions)



3: Usability (Fast, Simple Workflows)

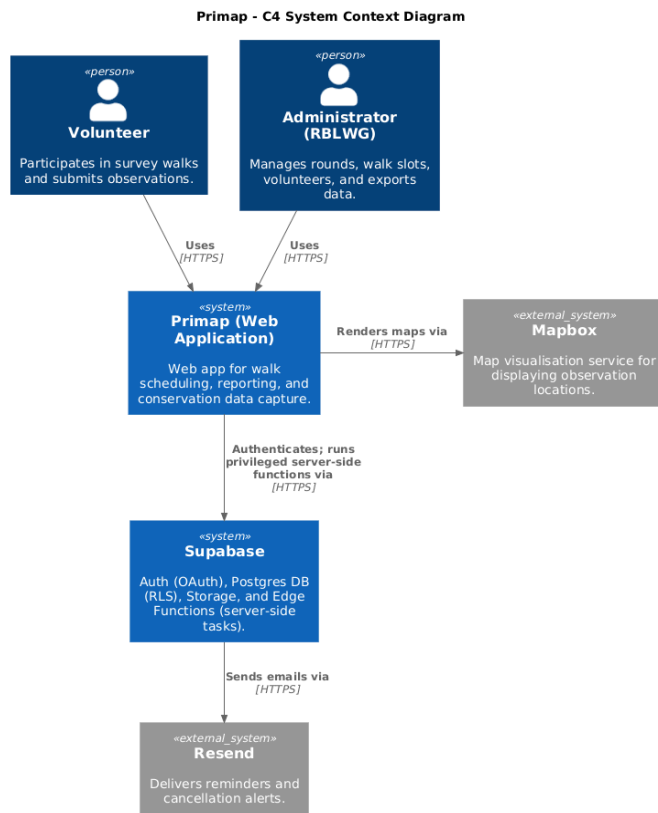
- **Architectural Driver:** $\geq 80\%$ first-time volunteers can sign up and submit within 10 minutes and slot list loads within 3 seconds for 95% of requests on mobile networks.
- **Approaches:**
 - **Option A (Server-rendered Flows with Minimal Client Caching):** Rely mostly on server rendering and direct DB fetch on every navigation.
 - **Option B (Client-optimised PWA UX with Caching and Background Refresh):** Use cached UI shell and cached slot lists. Implement background refresh when online and use stepwise forms.
- **Tradeoffs**
 - **Option A**
 - **Pros:** Less client state complexity
 - **Cons:** Worse performance offline/in poor network conditions; harder to implement offline draft workflows
 - **Option B**
 - **Pros:** Faster initial load (cached shell) which works better offline and allows easier implementation of stepwise forms
 - **Cons:** More client logic and state management
- **Resulting Decisions**

We will adopt **Option B (Client-optimised PWA UX with Caching and Background Refresh)** to quicken load times and improve user experience while simultaneously supporting offline workflows.

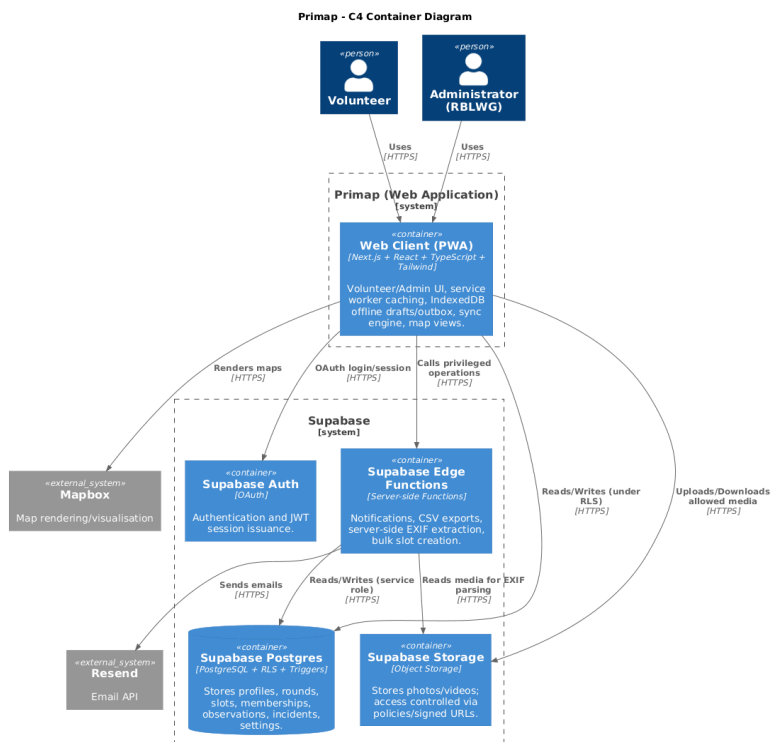


3.2. System Architecture Views

3.2.1. System Context Diagram



3.2.2. Container Diagram



3.3. Component Design

3.3.1. Container Decomposition

This section decomposes each major C4 container identified in the Container Diagram (Section 3.2.2) into its internal components, describing their responsibilities and key interactions.

Major Containers Identified

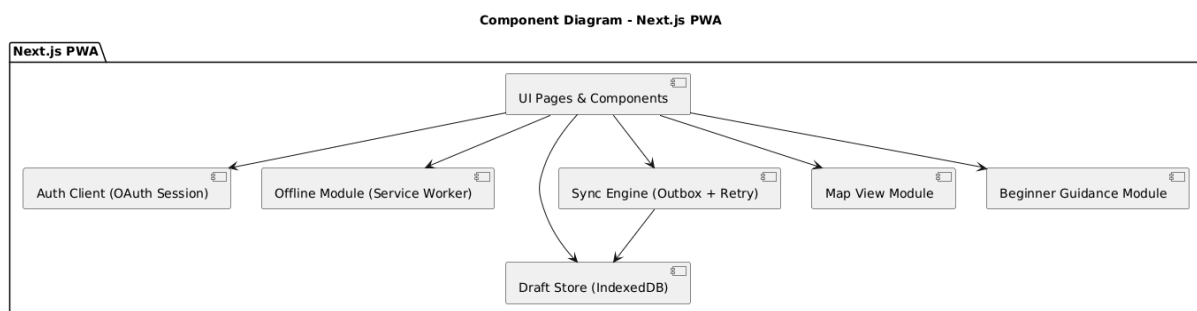
1. Next.js PWA (Frontend UI + Offline Modules)
2. Supabase Edge Functions
3. Supabase Postgres
4. Supabase Auth
5. Supabase Storage

3.3.1.1. Component Diagram: Next.js PWA Container

Components

- **UI Pages & Components:** Forms and dashboards for both volunteers and admins
- **Auth Client:** OAuth login flow and session handling
- **Offline Module:** Service worker caching for app shell and key routes and assets
- **Draft Store:** IndexedDB persistence for drafts and outbox queue
- **Sync Engine:** Retries queued operations, handles idempotency key and reports sync status
- **Map View:** Renders points for submitted data

Diagram



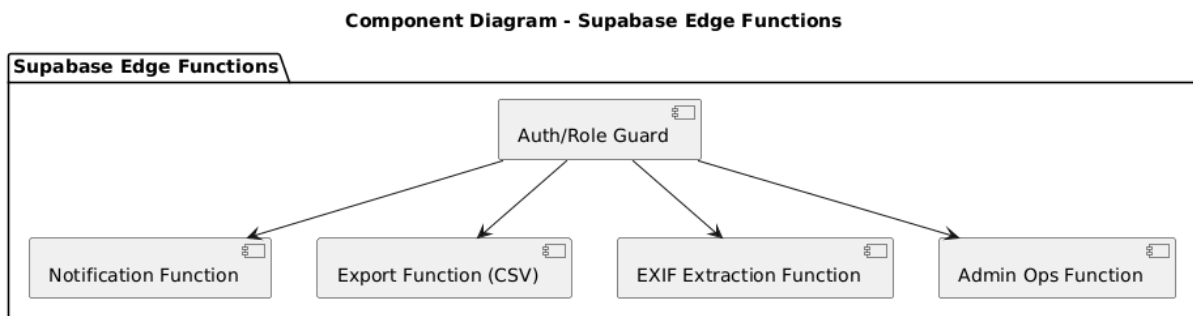
3.3.1.2. Component Diagram: Supabase Edge Functions Container

Components

- **Admin Ops Function:** Admin operations such as bulk slot creation
- **Auth/Authorisation Guard:** Verifies JWT and role before allowing admin actions
- **Export Function:** Generates CSV data exports from DB
- **Notification Function:** Manages email reminders and cancellation alerts

- **EXIF Extraction Function:** Extracts GPS/date-time from uploaded images

Diagram

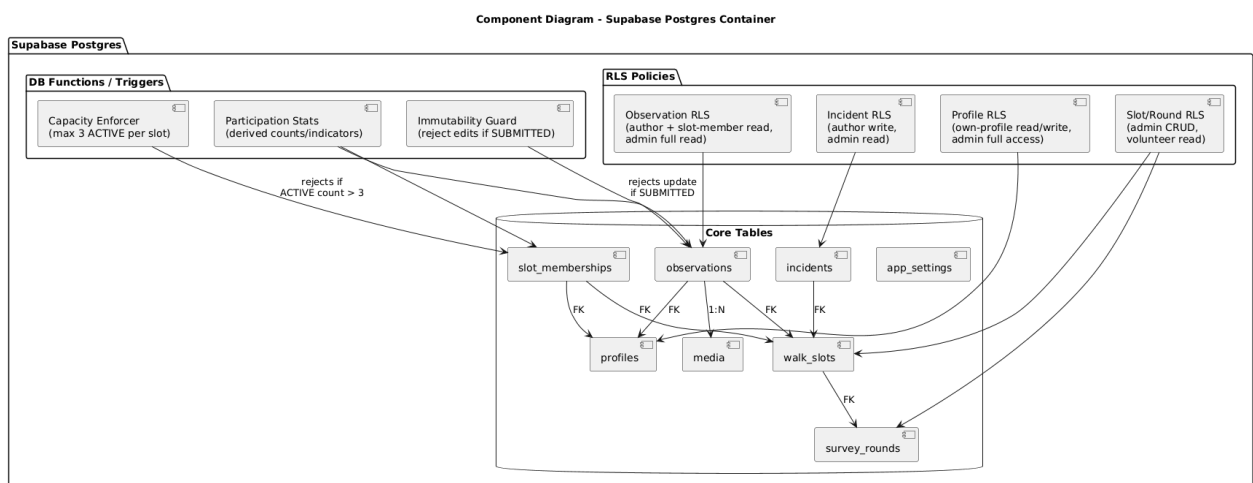


3.3.1.3. Component Diagram: Supabase Postgres Container

Components

- **Core Tables:** Stores all relational data including [profiles](#), [survey_rounds](#), [walk_slots](#), [slot_memberships](#), [observations](#), [media](#), [incidents](#), and [app_settings](#).
- **Profile RLS:** Enforces access rules for user profiles.
- **Slot/Round RLS:** Enforces access rules for survey slots.
- **Observation RLS:** Enforces access rules for submitted reports.
- **Incident RLS:** Enforces access rules for reporting incidents and incident reports.
- **Capacity Trigger:** Enforces capacity limits for walk slots.
- **Immutability Trigger:** Prevents editing of forms after submission.
- **Participation Stats Functions:** Computes volunteer analytics.

Diagram



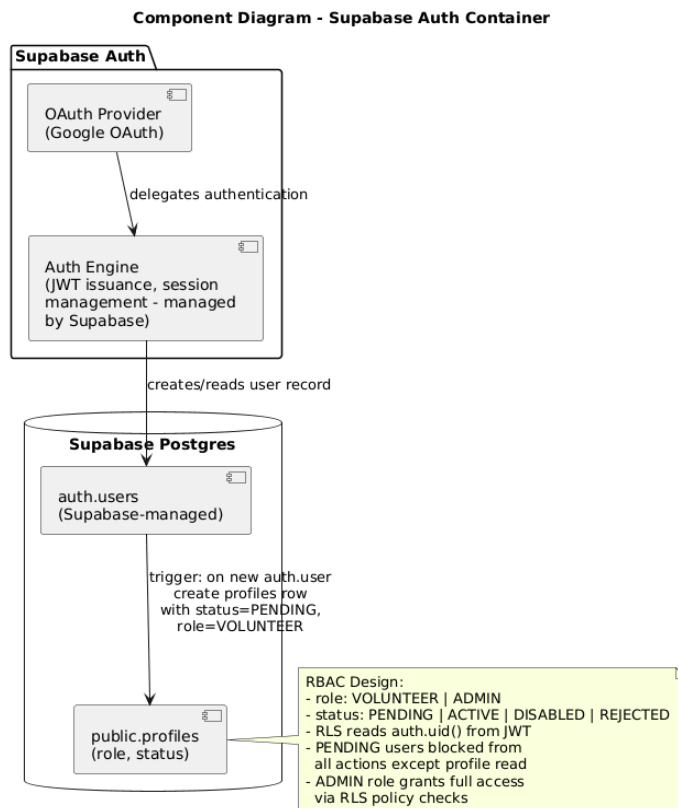
3.3.1.4. Component Diagram: Supabase Auth Container

Components

- **OAuth Provider (Google OAuth):** Handles third-party authentication flow, redirecting users to Google and receiving identity tokens.

- **Profile Creation Trigger:** Manages creation of new users in our own database.
- **Profiles Table:** Stores user profiles.

Diagram

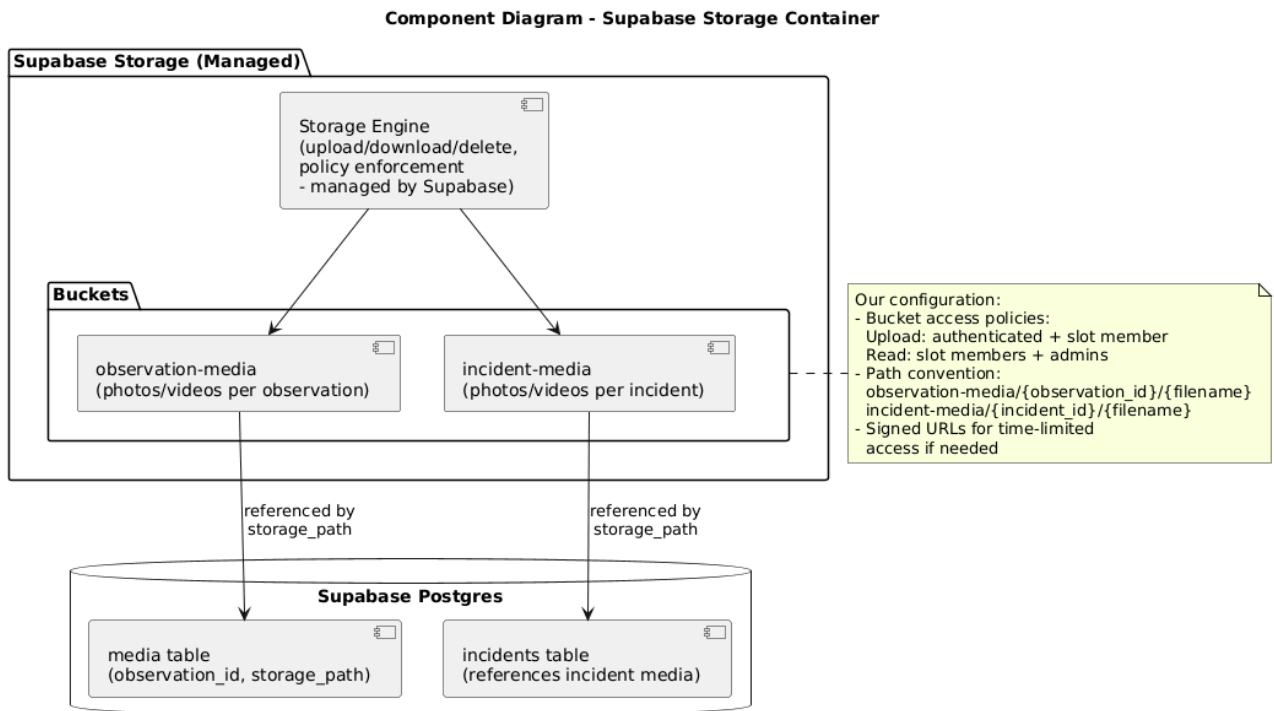


3.3.1.5. Component Diagram: Supabase Storage Container

Components

- **Observation-media bucket:** Stores photos and videos attached to observation reports.
- **Incident-media bucket:** Stores photos and videos attached to incident reports.
- **Bucket Access Policies:** Manages access policies for the buckets

Diagram



Sections 3.3.1 above decomposed each container into its internal components. However, in the following section, we look at each service as a feature and show how User Management, Walk Management, and Report Management services will function across multiple containers.

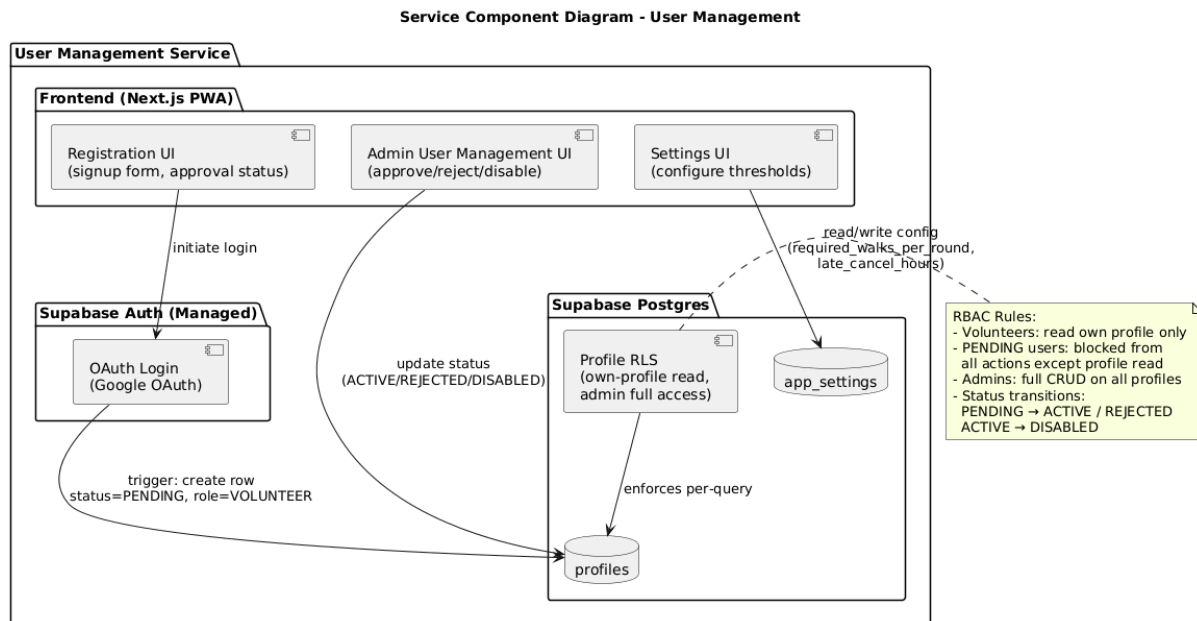
3.3.2. Service Design (Cross-Container View)

3.3.2.1. User Management Service/Component

Design Rationale

Primap uses **OAuth-based authentication** via Supabase Auth to avoid password management complexity for users. User role and status are stored in a **profiles** table, enabling an explicit approval workflow where each volunteer account has to be explicitly approved by an admin before they can take any action. This is easier to implement than, for example, an invitation based registration flow while ensuring that there is some level of data protection. Access control is enforced primarily through RLS, ensuring volunteers can only read or modify their own profile fields while admins can manage accounts system-wide. Edge Functions are used to check permissions for admin actions where appropriate (e.g., admin bulk operations).

Component Diagram



Data Structure Design

Table: **profiles**

- **id** (UUID, PK) — matches Supabase Auth user id
- **name** (text, non-empty) — volunteer/admin name
- **email** (text, unique) — for identification and notifications
- **role** (enum: **VOLUNTEER**, **ADMIN**) — role-based access
- **status** (enum: **PENDING**, **ACTIVE**, **DISABLED**, **REJECTED**) — approval and enforcement
- **created_at** (timestamp) — audit and operations
- **approved_at** (timestamp, nullable) — approval record

Table: **app_settings** (configurable thresholds/settings)

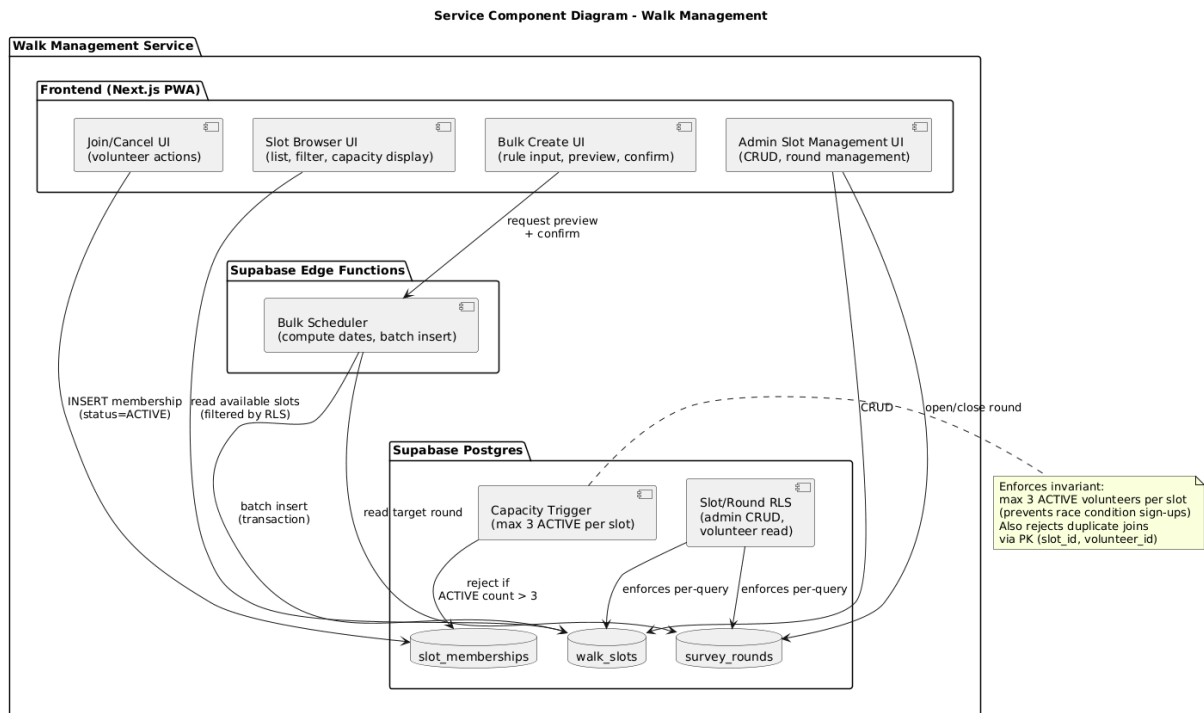
- **key** (text, PK) — e.g., **required_walks_per_round**, **late_cancel_hours**
- **value** (json/text) — stores configurable values
- **updated_at** (timestamp)
- **updated_by** (uuid, FK -> profiles)

3.3.2.2. Walk Management Service/Component

Design Rationale

Volunteers can browse upcoming slots and join or cancel, while admins can CRUD slots and bulk-create slots using scheduling rules for multiple weeks. Survey rounds and walk slots have explicit capacity constraints which are enforced with **database trigger/function** to ensure capacity limits are enforced at the database layer which would prevent accidental oversubscriptions more reliably. Security is enforced by RLS so only admins can create/update slots, while volunteers can only view slots relevant to them.

Component Diagram



Data Structure Design

Table: **survey_rounds**

- **id** (UUID, PK)
- **start_date** (date)
- **end_date** (date)
- **status** (enum: **OPEN**, **CLOSED**)
- **created_at** (timestamp)
- **created_by** (uuid, FK -> profiles)

Table: **walk_slots**

- **id** (UUID, PK)
- **round_id** (uuid, FK -> survey_rounds)
- **date** (date)
- **start_time** (time)
- **end_time** (time)
- **location_desc** (text, nullable)
- **capacity** (int, default 3, CHECK capacity ≤ 3)
- **created_at** (timestamp)
- **created_by** (uuid, FK -> profiles)

Table: **slot_memberships**

- **slot_id** (uuid, FK -> walk_slots)
- **volunteer_id** (uuid, FK -> profiles)
- **status** (enum: **ACTIVE**, **CANCELLED**)

- **joined_at** (timestamp)
- **cancelled_at** (timestamp, nullable)
- **Primary Key:** (**slot_id**, **volunteer_id**)

DB Trigger/Function (capacity enforcement)

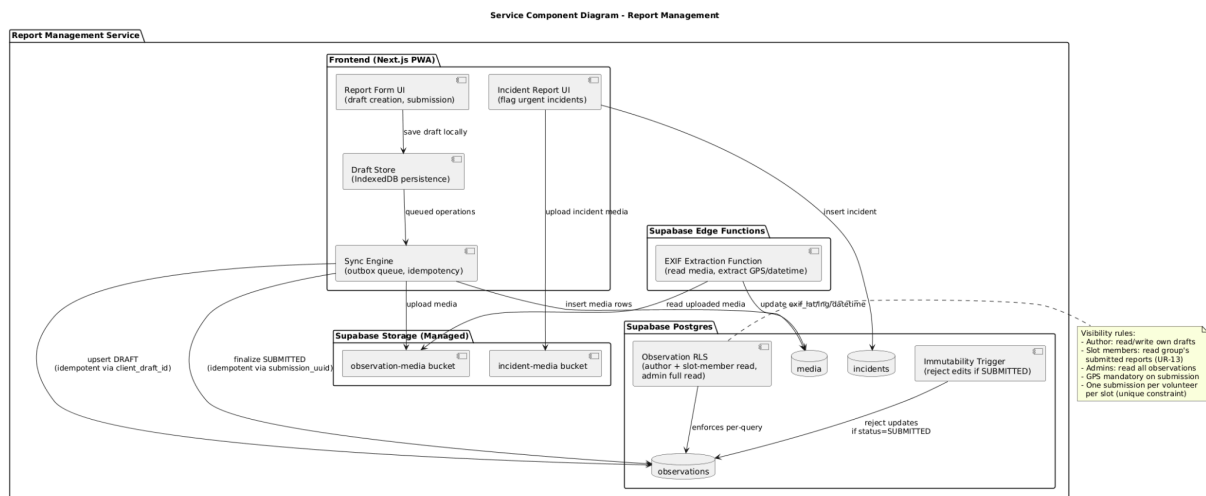
- On insert/update to **slot_memberships** setting **status=ACTIVE**, a trigger checks active count for that slot; rejects if count would exceed 3.

3.3.2.3. Report Management Service/Component

Design Rationale

Report management supports volunteer reporting of sightings for a walk slot, including “Not Sighted” outcomes. Each volunteer submits an **individual report** per slot, and the slot view aggregates all member submissions for group visibility (UR-13). Compared to allowing the whole group to submit one report, this reduces the number of edge cases and simplifies the flow of the system. It also makes handling the offline drafting feature simpler. Reports become **immutable after submission** enforced by database rules/constraints (UR-10) and will be visible only to volunteers who participated in the slot as per stated requirements.. Admins will be able to view all submitted reports to support their analyses. Media is stored in Supabase Storage and referenced in a **media** table. GPS metadata extraction from images is performed **server-side** to reduce client complexity and ensure consistent parsing of images.

Component Diagram



Data Structure Design

Table: **observations**

- **id** (UUID, PK)
- **slot_id** (uuid, FK -> walk_slots)
- **volunteer_id** (uuid, FK -> profiles)
- **client_draft_id** (uuid) — idempotency key for sync
- **status** (enum: **DRAFT**, **SUBMITTED**)
- **walk_completion** (enum: **COMPLETED**, **PARTIAL**, **ABORTED**, **UNRECORDED**)

- **outcome** (enum: **SIGHTED**, **NOT_SIGHTED**)
- **species** (enum: **RBL**, **LTM**, **DUSKY**, nullable; required if **SIGHTED**)
- **count** (text; supports “x-y”, “Unknown”, or numeric string)
- **notes** (text, nullable)
- **observed_at** (timestamp, nullable)
- **lat** (decimal, required at **SUBMITTED**)
- **lng** (decimal, required at **SUBMITTED**)
- **created_at** (timestamp)
- **submitted_at** (timestamp, nullable)
- **Unique constraint:** (**slot_id**, **volunteer_id**) — ensures one submitted report per volunteer per slot

Table: **media**

- **id** (UUID, PK)
- **observation_id** (uuid, FK -> observations)
- **type** (enum: **PHOTO**, **VIDEO**)
- **storage_path** (text)
- **exif_lat** (decimal, nullable)
- **exif_lng** (decimal, nullable)
- **exif_datetime** (timestamp, nullable)
- **created_at** (timestamp)

Table: **incidents**

- **id** (UUID, PK)
- **slot_id** (uuid, FK -> walk_slots)
- **volunteer_id** (uuid, FK -> profiles)
- **type** (enum: **TRAP**, **FEEDING**, **ROADKILL**, **OTHER**)
- **details** (text, nullable)
- **created_at** (timestamp)

DB Trigger/Function (immutability)

- Reject updates to key fields when **observations.status = SUBMITTED** (except admin-only corrective fields if desired).

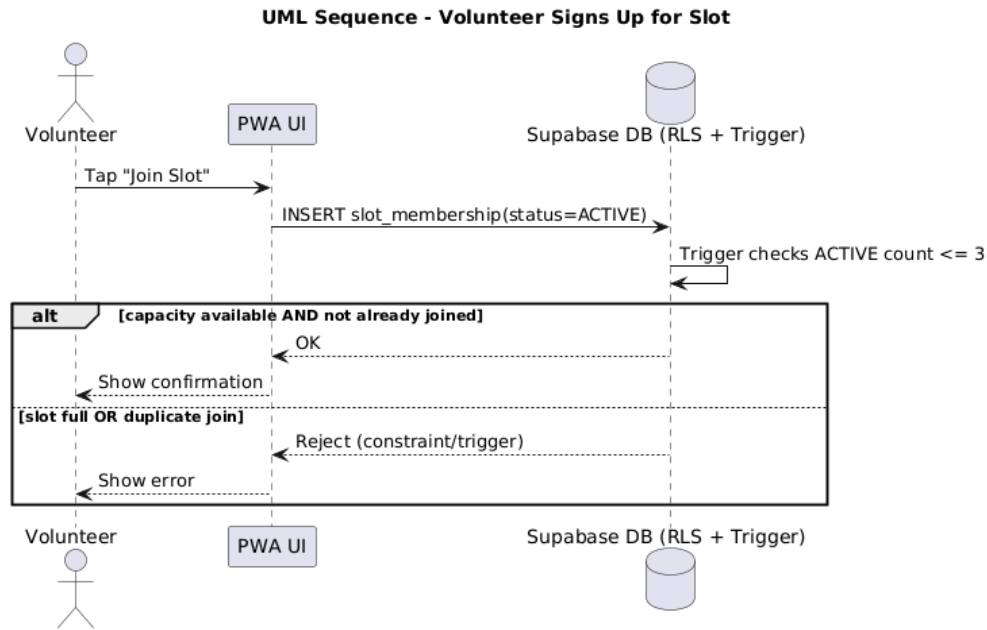
Map Visualisation Data

- Map points are sourced from **observations.lat/lng** and filtered by RLS:
 - Volunteers: see own points and slot-group points for slots they are members of
 - Admins: see broader sets such as per-volunteer or per-round views

3.3.3. Behavioral Diagrams (UML) – Three Important System Flows

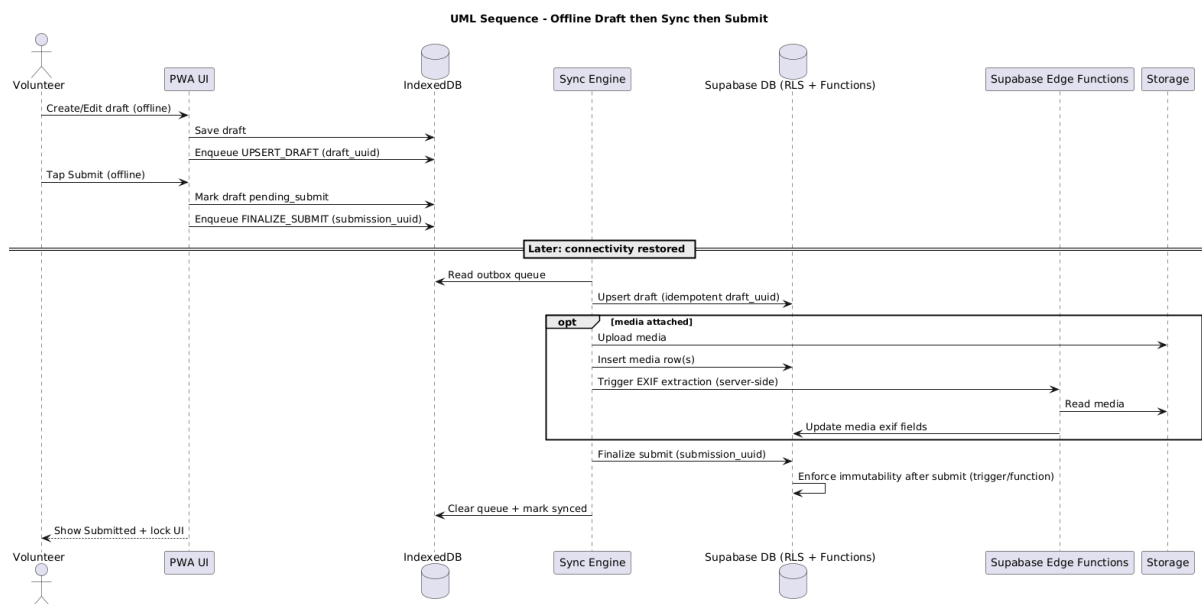
3.3.3.1. Flow 1: Volunteer signs up for a walk slot (capacity max 3)

Goal: enforce capacity and prevent duplicates.



3.3.3.2. Flow 2: Offline report drafting → later sync → final submission (immutable)

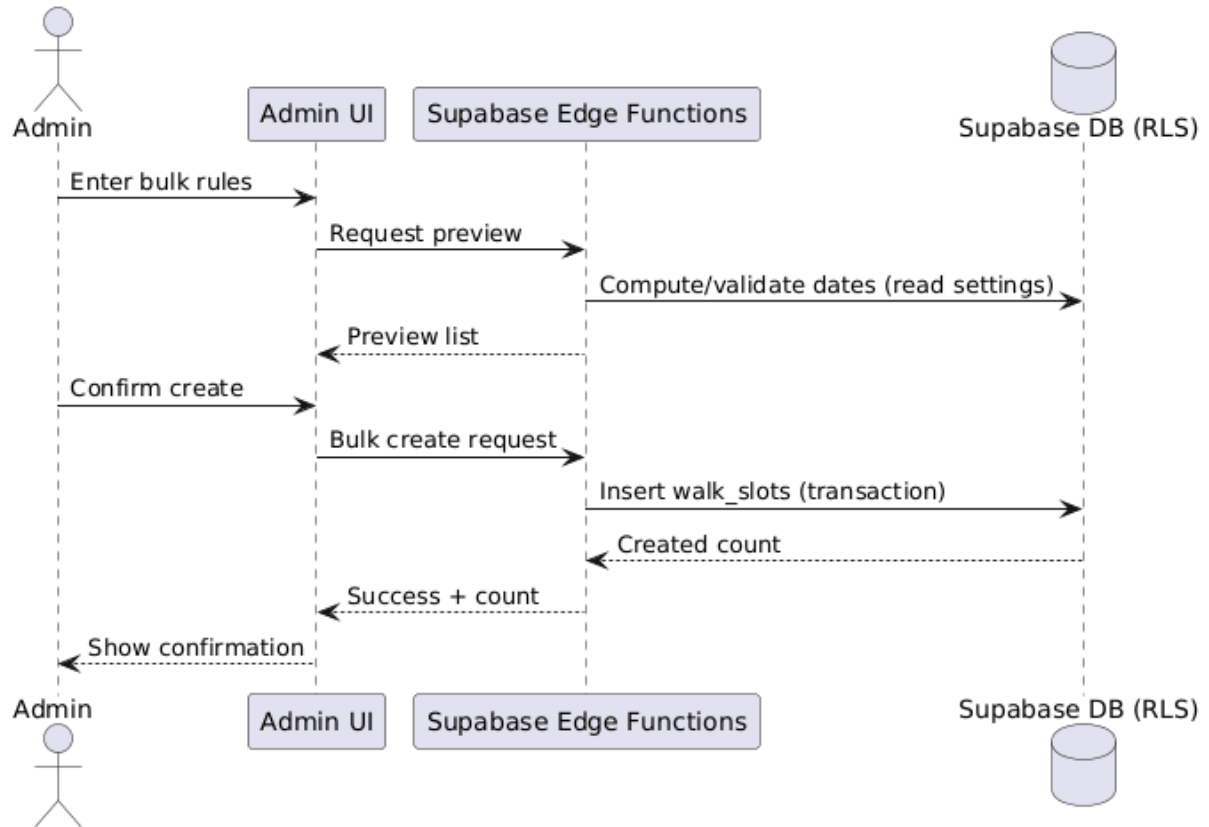
Goal: offline-first reliability and idempotent sync.



3.3.3.3. Flow 3: Admin bulk-creates walk slots based on rule pattern

Goal: efficient scheduling and preview-before-commit.

UML Sequence - Admin Bulk Creates Walk Slots



4. Technology Stack and Trade-offs

4.1. Selected Technologies

1. **Next.js (TypeScript) + Tailwind CSS**

Next.js provides a unified framework for the web UI and server-side API routes, enabling rapid development with shared TypeScript types. Tailwind CSS supports consistent, responsive UI with minimal custom CSS, improving maintainability and speed of iteration.

2. **Supabase (Auth + Postgres + Storage + Edge Function)**

Supabase provides OAuth authentication, a relational Postgres database, row-level security policies, and storage for media uploads. Postgres triggers/functions support rules such as slot capacity and immutability, and relational exports (CSV) are straightforward. Supabase Edge Functions provide server-side execution for tasks requiring secrets or server computation (e.g., email sending, server-side EXIF extraction, CSV export generation) without operating a separate backend server.

3. **PWA (Service Worker + IndexedDB)**

Service workers allow caching of the app shell so the application can be opened without network after initial load. IndexedDB supports offline draft persistence and queued operations for later synchronisation.

4. **Map Visualisation (Mapbox)**

Map visualisation is used to display observation points for volunteers and admins (e.g., per-volunteer/per-round exploration). Mapbox provides interactive map rendering and mapping utilities suitable for web applications.

5. **Email Notification Provider (Resend)**

Resend handles email reminders to volunteers to inform them of upcoming walks and cancellation alerts for admins when a volunteer withdraws from a slot.

4.2. Selection Rationale

- **Next.js (TypeScript) + Tailwind CSS**

Broadest ecosystem among React frameworks with plenty of documentation and help available online. Also, has plenty of support for PWAs. TypeScript provides shared type safety across client and server, reducing bugs in critical flows like offline sync. Tailwind provides sensible default CSS classes and is highly compatible with the component based structure of React.

- **Supabase (Auth + Postgres + Storage + Edge Functions)**

Our system requires strong relational constraints — slot capacity enforcement, unique memberships, immutable submissions — all naturally expressed in SQL with triggers and CHECK constraints, whereas Firestore would require application-level enforcement which could be more prone to errors. Furthermore, RLS provides access control aligned with our privacy requirements. Auth handles OAuth/JWT as a managed service, easing volunteer onboarding (UR-01). Edge Functions cover

privileged server-side operations without the overhead of a separate backend, reducing developmental costs and time.

- **PWA (Service Worker + IndexedDB)**

The system explicitly requires offline draft creation and reliable sync (UR-06, UR-09). Survey routes often have poor cellular coverage, making offline capability essential. Service workers enable app loading without network after first visit and IndexedDB provides draft storage and a queued outbox that LocalStorage cannot reliably support.

- **Mapbox**

Good balance of rendering quality, developer-friendly APIs, and reasonable pricing for a low-traffic application. Avoids Google Maps' cost/key management complexity and Leaflet's custom integration effort for clustering and styling.

- **Resend**

Simple API matching our limited email needs — walk reminders (UR-16) and cancellation alerts (UR-17). SendGrid's extensive feature set adds unnecessary complexity for low-volume transactional email. Additionally, Resend integrates nicely with Supabase Edge Functions.

4.3. Alternatives and Trade-offs

A) Supabase vs Firebase

- **Chosen:** Supabase
- **Alternative:** Firebase (Auth + Firestore + Storage)
 - **Pros:** Mature ecosystem, straightforward auth and storage integration
 - **Cons:** No relational database to support our database constraints and triggers

B) Edge Functions vs Dedicated Backend Server

- **Chosen:** Supabase Edge Functions
- **Alternative:** Separate backend server (Fastify/Express)
 - **Pros:** Full control over runtime and libraries and clear separation for complex business logic
 - **Cons:** Higher deployment and implementation complexity for authentication and authorization systems.

C) Mapbox vs Google Maps vs Leaflet/OSM

- **Chosen:** Mapbox
- **Alternative 1:** Google Maps
 - **Pros:** Familiar UX and strong POI/geocoding ecosystem
 - **Cons:** Pricing and quota considerations and API key management overhead

- **Alternative 2:** Leaflet + OpenStreetMap tiles
 - **Pros:** Open ecosystem, flexible, potentially lower cost
 - **Cons:** Higher integration effort and requires additional work for advanced features like clustering and styling

D) PWA Offline (Service Worker + IndexedDB) vs Simpler Offline Handling

- **Chosen:** Service Worker + IndexedDB
- **Alternative:** LocalStorage drafts
 - **Pros:** Simpler implementation
 - **Cons:** LocalStorage is limited and not suitable for robust queued operations

E) Resend vs SendGrid

- **Chosen:** Resend
- **Alternative:** Twilio, SendGrid
 - **Pros:** Very mature tooling that supports sending of a high volume of emails and lots of templates/integrations
 - **Cons:** More complicated to setup than Resend for a simple email flow and requires payment after 60 days

F) Next.js vs Remix

- **Chosen:** Next.js
- **Alternative:** Remix
 - **Pros:** Nested routing + data loading model, can be simpler to reason about request/response flows
 - **Cons:** Smaller ecosystem and less guidance than Next.js and fewer templates to use